

Programming Language Foundations Appunti

Alex "Morbidelli WhatsApp" Basta

Contents

Chapter 1	Operational Semantics	Page
1.1	Large-step Operational Semantics	
1.2	Equivalence of semantics	
1.3	A simple imperative language IMP Small-step operational semantics —	
1.4	Large-step operational semantics for IMP Command equivalence —	
Chapter 2	Denotational Semantics	Page
2.1	A Denotational Semantics for IMP	
2.2	Fixed Points	
2.3	Denotation for Loops	
2.4	Kleene's Fixed Point Theorem	
2.5	Reasoning using Denotational Semantics	
Chapter 3	Axiomatic Semantics	Page
3.1	Introduction to axiomatic semantics	
3.2	Assertions	
3.3	Satisfaction and Validity	
3.4	Hoare Logic	
3.5	Soundness and Completeness	
3.6	Example: Factorial	
3.7	Decorated Programs Informal Rules — • Decorating a Program —	
3.8	Weakest Preconditions	

Chapter 1

Operational Semantics

Binary relations, functions (total/partial), operational semantics

Derivation tree of an arithmetic expression using the inference rules. (slide 2)

Definition 1.0.1: Multi-step evaluation

Binary relation indicated with $\rightarrow^*$ which is the reflexive and transitive closure of the small-step evaluation.

Note that the transitive rule is a single step and a multi step, although it can be proved that it allows concatenation of single steps.

Properties of programs

- Determinism: suppose that for any given configuration, if this can step in different ways these are equal
- Termination: evaluation terminates. Formally, for the multi step there is a configuration that it can multi step to without other possible steps.
- Soundness: if we run a program long enough we should get a final integer, but it's not true because stores are partial function and if a variable isn't defined we're cooked.

Definition 1.0.2: Free Variables

Given the function $fvs(e)$ defined inductively as:

Note that in $x := x + 1; y := x$ we still need to add x to the free variables because it's in the rhs of the first assignment.

Definition 1.0.3: Well-Formedness

Example of an early type system. All the variables that e refers to have values in σ .

Two properties that imply soundness:

Lemma 1.0.1 Progress

Lemma 1.0.2 Preservation

with these lemmas and termination we can show that a well-formed configuration is sound.

A bit of maths before the proof

Definition 1.0.4: Inductive Set

We have used inductive sets with the inductive rules (remember that $\rightarrow$ is a relation, so a set of tuples) and BNF grammar, free-variables and multistep. Another example is the natural numbers using the successor definition.

If we want to prove something about an inductive set (that can be infinite by closing under the inductive rules) we can use the induction principle.

The base cases are all the axioms.

Sometimes the properties are defined over multiple inductive sets, so we need to choose one.

1.1 Large-step Operational Semantics

In the last lecture we defined a semantics for our language of arithmetic expressions using a small-step evaluation relation $\rightarrow \subseteq \mathbf{Config} \times \mathbf{Config}$ (and its reflexive and transitive closure $\rightarrow^*$). In this lecture we will explore an alternative approach—*large-step* operational semantics—which yields the final result of evaluating an expression directly.

Defining a large-step semantics boils down to specifying a relation $\Downarrow$ that captures the evaluation of an expression. The $\Downarrow$ relation has the following type:

$$\Downarrow \subseteq (\mathbf{Store} \times \mathbf{Exp}) \times (\mathbf{Store} \times \mathbf{Int}).$$

We write $\langle \sigma, e \rangle \Downarrow \langle \sigma', n \rangle$ to indicate that $((\sigma, e), (\sigma', n)) \in \Downarrow$. In other words, the expression e with store σ evaluates in one big step to the final store σ' and integer n .

We define the relation $\Downarrow$ inductively, using inference rules:

$$\frac{}{\langle \sigma, n \rangle \Downarrow \langle \sigma, n \rangle} \text{INT} \quad \frac{n = \sigma(x)}{\langle \sigma, x \rangle \Downarrow \langle \sigma, n \rangle} \text{VAR}$$

Note:

Note that in the big step semantics, we need an inference rule for every valid configuration, hence the INT rule.

$$\frac{\langle \sigma, e_1 \rangle \Downarrow \langle \sigma', n_1 \rangle \quad \langle \sigma', e_2 \rangle \Downarrow \langle \sigma'', n_2 \rangle \quad n = n_1 + n_2}{\langle \sigma, e_1 + e_2 \rangle \Downarrow \langle \sigma'', n \rangle} \text{ADD}$$

$$\frac{\langle \sigma, e_1 \rangle \Downarrow \langle \sigma', n_1 \rangle \quad \langle \sigma', e_2 \rangle \Downarrow \langle \sigma'', n_2 \rangle \quad n = n_1 \times n_2}{\langle \sigma, e_1 * e_2 \rangle \Downarrow \langle \sigma'', n \rangle} \text{MUL}$$

$$\frac{\langle \sigma, e_1 \rangle \Downarrow \langle \sigma', n_1 \rangle \quad \langle \sigma'[x \mapsto n_1], e_2 \rangle \Downarrow \langle \sigma'', n_2 \rangle}{\langle \sigma, x := e_1; e_2 \rangle \Downarrow \langle \sigma'', n_2 \rangle} \text{ASSGN}$$

Example 1.1.1 (Big-step proof)

To illustrate the use of these rules, consider the following proof tree, which shows that evaluating $\langle \sigma, \text{foo} := 3; \text{foo} * \text{bar} \rangle$ using a store σ such that $\sigma(\text{bar}) = 7$ yields $\sigma' = \sigma[\text{foo} \mapsto 3]$ and 21 as a result:

$$\frac{\frac{}{\langle \sigma, 3 \rangle \Downarrow \langle \sigma, 3 \rangle} \text{INT} \quad \frac{\frac{}{\langle \sigma', \text{foo} \rangle \Downarrow \langle \sigma', 3 \rangle} \text{VAR} \quad \frac{}{\langle \sigma', \text{bar} \rangle \Downarrow \langle \sigma', 7 \rangle} \text{VAR}}{\langle \sigma', \text{foo} * \text{bar} \rangle \Downarrow \langle \sigma', 21 \rangle} \text{MUL}}{\langle \sigma, \text{foo} := 3; \text{foo} * \text{bar} \rangle \Downarrow \langle \sigma', 21 \rangle} \text{ASSGN}$$

A closer look to this structure reveals the relation between small step and large-step evaluation: a depth-first traversal of the large-step proof tree yields the sequence of one-step transitions in small-step evaluation.

Note:

Big step semantics don't capture the computational intricacies (if not well-formed it's undefined instead of blocking), and it struggles with concurrency. For this reason it can be used together with small step for the critical parts.

Note:

Also, small step semantics can be used to precisely count computation ticks (cslib 4 lean) for tighter complexity bounds.

1.2 Equivalence of semantics

A natural question to ask is whether the small-step and large-step semantics are equivalent. The next theorem answers this question affirmatively.

Theorem 1.2.1 Equivalence of semantics

For all expressions e , stores σ and σ' , and integers n we have:

$$\langle \sigma, e \rangle \Downarrow \langle \sigma', n \rangle \quad \text{if and only if} \quad \langle \sigma, e \rangle \rightarrow^* \langle \sigma', n \rangle$$

To streamline the proof, we will work with the following definition of the multi-step relation:

$$\frac{}{\langle \sigma, e \rangle \rightarrow^* \langle \sigma, e \rangle} \text{REFL} \quad \frac{\langle \sigma, e \rangle \rightarrow \langle \sigma', e' \rangle \quad \langle \sigma', e' \rangle \rightarrow^* \langle \sigma'', e'' \rangle}{\langle \sigma, e \rangle \rightarrow^* \langle \sigma'', e'' \rangle} \text{TRANS}$$

Proof Sketch: We show each direction separately.

$\Rightarrow$: We want to prove that the following property P holds for all expressions $e \in \mathbf{Exp}$:

$$P(e) \triangleq \forall \sigma, \sigma' \in \mathbf{Store}. \forall n \in \mathbf{Int}. \langle \sigma, e \rangle \Downarrow \langle \sigma', n \rangle \implies \langle \sigma, e \rangle \rightarrow^* \langle \sigma', n \rangle$$

We proceed by structural induction on e . We have to consider each of the possible axioms and inference rules for constructing an expression.

Case $e = x$: Assume that $\langle \sigma, x \rangle \Downarrow \langle \sigma', n \rangle$. That is, there is some derivation in the large-step operational semantics whose conclusion is $\langle \sigma, x \rangle \Downarrow \langle \sigma', n \rangle$. There is only one rule whose conclusion matches the configuration $\langle \sigma, x \rangle$: the large-step rule VAR. Thus, we have $n = \sigma(x)$ and $\sigma' = \sigma$. By the small-step rule VAR, we also have $\langle \sigma, x \rangle \rightarrow \langle \sigma, n \rangle$. By the REFL and TRANS rules, we conclude that $\langle \sigma, x \rangle \rightarrow^* \langle \sigma, n \rangle$, which finishes the case.

Case $e = n$: Assume that $\langle \sigma, n \rangle \Downarrow \langle \sigma', n' \rangle$. There is only one rule whose conclusion matches $\langle \sigma, n \rangle$: the large-step rule INT. Thus, we have $n' = n$ and $\sigma' = \sigma$ and so $\langle \sigma, n \rangle \rightarrow^* \langle \sigma, n \rangle$ by the REFL rule.

Case $e = e_1 + e_2$: This is an inductive case. We want to prove that if $P(e_1)$ and $P(e_2)$ hold, then $P(e)$ also holds. Let's write out $P(e_1)$, $P(e_2)$, and $P(e)$ explicitly.

$$\begin{aligned} P(e_1) &= \forall n, \sigma, \sigma'. \langle \sigma, e_1 \rangle \Downarrow \langle \sigma', n \rangle \implies \langle \sigma, e_1 \rangle \rightarrow^* \langle \sigma', n \rangle \\ P(e_2) &= \forall n, \sigma, \sigma'. \langle \sigma, e_2 \rangle \Downarrow \langle \sigma', n \rangle \implies \langle \sigma, e_2 \rangle \rightarrow^* \langle \sigma', n \rangle \\ P(e) &= \forall n, \sigma, \sigma'. \langle \sigma, e_1 + e_2 \rangle \Downarrow \langle \sigma', n \rangle \implies \langle \sigma, e_1 + e_2 \rangle \rightarrow^* \langle \sigma', n \rangle \end{aligned}$$

Assume that $P(e_1)$ and $P(e_2)$ hold. Also assume that there exist σ, σ' and n such that $\langle \sigma, e_1 + e_2 \rangle \Downarrow \langle \sigma', n \rangle$. We need to show that $\langle \sigma, e_1 + e_2 \rangle \rightarrow^* \langle \sigma', n \rangle$.

We assumed that $\langle \sigma, e_1 + e_2 \rangle \Downarrow \langle \sigma', n \rangle$. This means that there is some derivation whose conclusion is $\langle \sigma, e_1 + e_2 \rangle \Downarrow \langle \sigma', n \rangle$. By inspection, we see that only one rule has a conclusion of this form: the ADD rule. Thus, the last rule used in the derivation was ADD and it must be the case that $\langle \sigma, e_1 \rangle \Downarrow \langle \sigma'', n_1 \rangle$ and $\langle \sigma'', e_2 \rangle \Downarrow \langle \sigma', n_2 \rangle$ hold for some n_1 and n_2 with $n = n_1 + n_2$.

By the induction hypothesis $P(e_1)$, as $\langle \sigma, e_1 \rangle \Downarrow \langle \sigma'', n_1 \rangle$, we must have $\langle \sigma, e_1 \rangle \rightarrow^* \langle \sigma'', n_1 \rangle$. Likewise, by induction hypothesis $P(e_2)$, we have $\langle \sigma'', e_2 \rangle \rightarrow^* \langle \sigma', n_2 \rangle$. By Lemma 1 below, we have,

$$\langle \sigma, e_1 + e_2 \rangle \rightarrow^* \langle \sigma'', n_1 + e_2 \rangle,$$

and by another application of Lemma 1 we have:

$$\langle \sigma'', n_1 + e_2 \rangle \rightarrow^* \langle \sigma', n_1 + n_2 \rangle$$

Then, using the small-step ADD rule and the multi-step TRANS rule, we have:

$$\frac{\frac{n = n_1 + n_2}{\langle \sigma', n_1 + n_2 \rangle \rightarrow \langle \sigma', n \rangle} \text{ADD} \quad \frac{}{\langle \sigma', n \rangle \rightarrow^* \langle \sigma', n \rangle} \text{REFL}}{\langle \sigma', n_1 + n_2 \rangle \rightarrow^* \langle \sigma', n \rangle} \text{TRANS}$$

Finally, by two applications of Lemma 2, we obtain $\langle \sigma, e_1 + e_2 \rangle \rightarrow^* \langle \sigma', n \rangle$, which finishes the case.

Case $e = e_1 * e_2$. Similar to case for $e_1 + e_2$ above.

Case $e = x := e_1; e_2$. Omitted. Try it as an exercise.

$\Leftarrow$: We proceed by induction on the derivation of $\langle \sigma, e \rangle \rightarrow^* \langle \sigma', n \rangle$ with a case analysis on the last rule used.

Case Refl: Then $e = n$ and $\sigma' = \sigma$. We immediately have $\langle \sigma, n \rangle \Downarrow \langle \sigma, n \rangle$ by the large-step rule INT.

Case Trans: Then $\langle \sigma, e \rangle \rightarrow \langle \sigma'', e'' \rangle$ and $\langle \sigma'', e'' \rangle \rightarrow^* \langle \sigma', n \rangle$. In this case, the induction hypothesis gives $\langle \sigma'', e'' \rangle \Downarrow \langle \sigma', n \rangle$. The result follows from Lemma 3 below. $\square$

Lenma 1.2.1

If $\langle \sigma, e \rangle \rightarrow^* \langle \sigma', n \rangle$, then the following hold:

- $\langle \sigma, e + e_2 \rangle \rightarrow^* \langle \sigma', n + e_2 \rangle$
- $\langle \sigma, e * e_2 \rangle \rightarrow^* \langle \sigma', n * e_2 \rangle$
- $\langle \sigma, n_1 + e \rangle \rightarrow^* \langle \sigma', n_1 + n \rangle$
- $\langle \sigma, n_1 * e \rangle \rightarrow^* \langle \sigma', n_1 * n \rangle$
- $\langle \sigma, x := e; e_2 \rangle \rightarrow^* \langle \sigma', x := n; e_2 \rangle$

: Omitted; try it as an exercise.

**Lenma 1.2.2**

If $\langle \sigma, e \rangle \rightarrow^* \langle \sigma', e' \rangle$ and $\langle \sigma', e' \rangle \rightarrow^* \langle \sigma'', e'' \rangle$, then $\langle \sigma, e \rangle \rightarrow^* \langle \sigma'', e'' \rangle$.

Lenma 1.2.3

Omitted; try it as an exercise.

Lenma 1.2.4

If $\langle \sigma, e \rangle \rightarrow \langle \sigma'', e'' \rangle$ and $\langle \sigma'', e'' \rangle \Downarrow \langle \sigma', n \rangle$, then $\langle \sigma, e \rangle \Downarrow \langle \sigma', n \rangle$.

Proof. Omitted; try it as an exercise.



1.3 A simple imperative language IMP

We will now consider a more realistic programming language, one where we can assign values to variables and execute control constructs such as **if** and **while**. The syntax for this imperative language, called IMP, is as follows:

$$\begin{array}{lll}
 \text{arithmetic expressions} & a \in \mathbf{Aexp} & a ::= x \mid n \mid a_1 + a_2 \mid a_1 \times a_2 \\
 \text{Boolean expressions} & b \in \mathbf{Bexp} & b ::= \mathbf{true} \mid \mathbf{false} \mid a_1 < a_2 \\
 \text{commands} & c \in \mathbf{Com} & c ::= \mathbf{skip} \mid x := a \mid c_1; c_2 \mid \mathbf{if } b \mathbf{ then } c_1 \mathbf{ else } c_2 \mid \mathbf{while } b \mathbf{ do } c
 \end{array}$$

1.3.1 Small-step operational semantics

We'll first give a small-step operational semantics for IMP. The configurations in this language are of the form $\langle \sigma, c \rangle$, $\langle \sigma, b \rangle$, and $\langle \sigma, a \rangle$, where σ is a store. The final configurations are of the form $\langle \sigma, \mathbf{skip} \rangle$ for commands, $\langle \sigma, \mathbf{true} \rangle$ and $\langle \sigma, \mathbf{false} \rangle$ for Boolean expressions, and $\langle \sigma, n \rangle$ for arithmetic expressions. There are three different small-step operational semantics relations: one each of the syntactic categories.

$$\rightarrow_{\mathbf{Com}} \subseteq (\mathbf{Store} \times \mathbf{Com}) \times (\mathbf{Store} \times \mathbf{Com})$$

$$\rightarrow_{\mathbf{Bexp}} \subseteq (\mathbf{Store} \times \mathbf{Bexp}) \times (\mathbf{Store} \times \mathbf{Bexp})$$

$$\rightarrow_{\mathbf{Aexp}} \subseteq (\mathbf{Store} \times \mathbf{Aexp}) \times (\mathbf{Store} \times \mathbf{Aexp})$$

For brevity, we will overload the symbol $\rightarrow$ and use it to refer to all of these relations. Which relation is being used will be clear from context. The evaluation rules for arithmetic and Boolean expressions are similar to the ones we've seen before. However, note that since the arithmetic expressions no longer contain assignment, arithmetic and Boolean expressions can not update the store. But to streamline everything we keep it as an output. In the small-step we'll fix the fact that the store doesn't change, but this can be altered if needed using σ' .

Arithmetic expressions

$$\begin{array}{c}
 \frac{n = \sigma(x)}{\langle \sigma, x \rangle \rightarrow \langle \sigma, n \rangle} \\
 \\
 \frac{\langle \sigma, a_1 \rangle \rightarrow \langle \sigma, a'_1 \rangle}{\langle \sigma, a_1 + a_2 \rangle \rightarrow \langle \sigma, a'_1 + a_2 \rangle} \quad \frac{\langle \sigma, a_2 \rangle \rightarrow \langle \sigma, a'_2 \rangle}{\langle \sigma, n + a_2 \rangle \rightarrow \langle \sigma, n + a'_2 \rangle} \quad \frac{p = n + m}{\langle \sigma, n + m \rangle \rightarrow \langle \sigma, p \rangle} \\
 \\
 \frac{\langle \sigma, a_1 \rangle \rightarrow \langle \sigma, a'_1 \rangle}{\langle \sigma, a_1 \times a_2 \rangle \rightarrow \langle \sigma, a'_1 \times a_2 \rangle} \quad \frac{\langle \sigma, a_2 \rangle \rightarrow \langle \sigma, a'_2 \rangle}{\langle \sigma, n \times a_2 \rangle \rightarrow \langle \sigma, n \times a'_2 \rangle} \quad \frac{p = n \times m}{\langle \sigma, n \times m \rangle \rightarrow \langle \sigma, p \rangle}
 \end{array}$$

Boolean expressions

$$\begin{array}{c}
 \frac{\langle \sigma, a_1 \rangle \rightarrow \langle \sigma, a'_1 \rangle}{\langle \sigma, a_1 < a_2 \rangle \rightarrow \langle \sigma, a'_1 < a_2 \rangle} \quad \frac{\langle \sigma, a_2 \rangle \rightarrow \langle \sigma, a'_2 \rangle}{\langle \sigma, n < a_2 \rangle \rightarrow \langle \sigma, n < a'_2 \rangle} \\
 \\
 \frac{n < m}{\langle \sigma, n < m \rangle \rightarrow \langle \sigma, \mathbf{true} \rangle} \quad \frac{n \geq m}{\langle \sigma, n < m \rangle \rightarrow \langle \sigma, \mathbf{false} \rangle}
 \end{array}$$

Commands

$$\begin{array}{c}
 \frac{\langle \sigma, a \rangle \rightarrow \langle \sigma, a' \rangle}{\langle \sigma, x := a \rangle \rightarrow \langle \sigma, x := a' \rangle} \quad \frac{}{\langle \sigma, x := n \rangle \rightarrow \langle \sigma[x \mapsto n], \mathbf{skip} \rangle} \\
 \\
 \frac{\langle \sigma, c_1 \rangle \rightarrow \langle \sigma', c'_1 \rangle}{\langle \sigma, c_1; c_2 \rangle \rightarrow \langle \sigma', c'_1; c_2 \rangle} \quad \frac{}{\langle \sigma, \mathbf{skip}; c_2 \rangle \rightarrow \langle \sigma, c_2 \rangle}
 \end{array}$$

For **if** commands, we reduce the test until we get **true** or **false** and then we execute the appropriate branch:

$$\frac{\langle \sigma, b \rangle \rightarrow \langle \sigma, b' \rangle}{\langle \sigma, \mathbf{if } b \mathbf{ then } c_1 \mathbf{ else } c_2 \rangle \rightarrow \langle \sigma, \mathbf{if } b' \mathbf{ then } c_1 \mathbf{ else } c_2 \rangle}$$

$$\overline{\langle \sigma, \text{if true then } c_1 \text{ else } c_2 \rangle} \rightarrow \langle \sigma, c_1 \rangle \quad \overline{\langle \sigma, \text{if false then } c_1 \text{ else } c_2 \rangle} \rightarrow \langle \sigma, c_2 \rangle$$

For **while** loops, the above strategy doesn't work, because if we evaluate b we won't be able to re-evaluate it for the next loop. Instead, we use the following rule, which can be thought of as “unrolling” the loop, one iteration at a time.

$$\overline{\langle \sigma, \text{while } b \text{ do } c \rangle} \rightarrow \langle \sigma, \text{if } b \text{ then } (c; \text{while } b \text{ do } c) \text{ else skip} \rangle$$

Note:

With this new rule, induction on the structure of the term doesn't work! This is because instead of always getting smaller, the step is to a bigger c . Sometimes this is manageable, but not always.

We can now take a concrete program and see how it executes under the above rules. Consider we execute the program

Example 1.3.1 (Small-step eval of IMP)

$foo := 3; \text{ while } foo < 4 \text{ do } foo := foo + 5$

The execution works as follows:

$$\begin{aligned} & \langle \sigma, foo := 3; \text{ while } foo < 4 \text{ do } foo := foo + 5 \rangle \\ \rightarrow & \langle \sigma', \text{skip}; \text{ while } foo < 4 \text{ do } foo := foo + 5 \rangle && \text{where } \sigma' = \sigma[foo \mapsto 3] \\ \rightarrow & \langle \sigma', \text{while } foo < 4 \text{ do } foo := foo + 5 \rangle \\ \rightarrow & \langle \sigma', \text{if } foo < 4 \text{ then } (foo := foo + 5; W) \text{ else skip} \rangle \\ \rightarrow & \langle \sigma', \text{if } 3 < 4 \text{ then } (foo := foo + 5; W) \text{ else skip} \rangle \\ \rightarrow & \langle \sigma', \text{if true then } (foo := foo + 5; W) \text{ else skip} \rangle \\ \rightarrow & \langle \sigma', foo := foo + 5; \text{while } foo < 4 \text{ do } foo := foo + 5 \rangle \\ \rightarrow & \langle \sigma', foo := 3 + 5; \text{while } foo < 4 \text{ do } foo := foo + 5 \rangle \\ \rightarrow & \langle \sigma', foo := 8; \text{while } foo < 4 \text{ do } foo := foo + 5 \rangle \\ \rightarrow & \langle \sigma'', \text{skip}; \text{while } foo < 4 \text{ do } foo := foo + 5 \rangle && \text{where } \sigma'' = \sigma'[foo \mapsto 8] \\ \rightarrow & \langle \sigma'', \text{while } foo < 4 \text{ do } foo := foo + 5 \rangle \\ \rightarrow & \langle \sigma'', \text{if } foo < 4 \text{ then } (foo := foo + 5; W) \text{ else skip} \rangle \\ \rightarrow & \langle \sigma'', \text{if } 8 < 4 \text{ then } (foo := foo + 5; W) \text{ else skip} \rangle \\ \rightarrow & \langle \sigma'', \text{if false then } (foo := foo + 5; W) \text{ else skip} \rangle \\ \rightarrow & \langle \sigma'', \text{skip} \rangle \end{aligned}$$

where W is an abbreviation for the while loop $\text{while } foo < 4 \text{ do } foo := foo + 5$.

1.4 Large-step operational semantics for IMP

We define large-step evaluation relations for arithmetic expressions, Boolean expressions, and commands. The relation for arithmetic expressions relates an arithmetic expression and store to the integer value that the expression evaluates to. For Boolean expressions, the final value is in **Bool** = {**true**, **false**}. For commands, the final value is a store.

$$\Downarrow_{\text{Aexp}} \subseteq (\text{Aexp} \times \text{Store}) \times \text{Int}$$

$$\Downarrow_{\text{Bexp}} \subseteq (\text{Bexp} \times \text{Store}) \times \text{Bool}$$

$$\Downarrow_{\text{Com}} \subseteq (\text{Com} \times \text{Store}) \times \text{Store}$$

Again, we overload the symbol $\Downarrow$ and use it for any of these three relations; which relation is intended will be clear from context. We also use infix notation, for example writing $\langle \sigma, c \rangle \Downarrow \sigma'$ if $(\langle \sigma, c \rangle, \sigma') \in \Downarrow_{\text{Com}}$.

Note:

In this course, we're not interested in concrete syntax, so ambiguous AST aren't a problem. Braces can be used to explicit.

Arithmetic expressions.

$$\frac{}{\langle \sigma, n \rangle \Downarrow n} \quad \frac{\sigma(x) = n}{\langle \sigma, x \rangle \Downarrow n}$$

$$\frac{\langle \sigma, a_1 \rangle \Downarrow n_1 \quad \langle \sigma, a_2 \rangle \Downarrow n_2 \quad n = n_1 + n_2}{\langle \sigma, a_1 + a_2 \rangle \Downarrow n} \quad \frac{\langle \sigma, a_1 \rangle \Downarrow n_1 \quad \langle \sigma, a_2 \rangle \Downarrow n_2 \quad n = n_1 \times n_2}{\langle \sigma, a_1 \times a_2 \rangle \Downarrow n}$$

Boolean expressions.

$$\frac{}{\langle \sigma, \text{true} \rangle \Downarrow \text{true}} \quad \frac{}{\langle \sigma, \text{false} \rangle \Downarrow \text{false}}$$

$$\frac{\langle \sigma, a_1 \rangle \Downarrow n_1 \quad \langle \sigma, a_2 \rangle \Downarrow n_2 \quad n_1 < n_2}{\langle \sigma, a_1 < a_2 \rangle \Downarrow \text{true}} \quad \frac{\langle \sigma, a_1 \rangle \Downarrow n_1 \quad \langle \sigma, a_2 \rangle \Downarrow n_2 \quad n_1 \geq n_2}{\langle \sigma, a_1 < a_2 \rangle \Downarrow \text{false}}$$

Commands.

$$\frac{}{\langle \sigma, \text{skip} \rangle \Downarrow \sigma} \text{ SKIP} \quad \frac{\langle \sigma, a \rangle \Downarrow n}{\langle \sigma, x := a \rangle \Downarrow \sigma[x \mapsto n]} \text{ ASSGN}$$

$$\frac{\langle \sigma, c_1 \rangle \Downarrow \sigma' \quad \langle \sigma', c_2 \rangle \Downarrow \sigma''}{\langle \sigma, c_1; c_2 \rangle \Downarrow \sigma''} \text{ SEQ}$$

$$\frac{\langle \sigma, b \rangle \Downarrow \text{true} \quad \langle \sigma, c_1 \rangle \Downarrow \sigma'}{\langle \sigma, \text{if } b \text{ then } c_1 \text{ else } c_2 \rangle \Downarrow \sigma'} \text{ IF-T} \quad \frac{\langle \sigma, b \rangle \Downarrow \text{false} \quad \langle \sigma, c_2 \rangle \Downarrow \sigma'}{\langle \sigma, \text{if } b \text{ then } c_1 \text{ else } c_2 \rangle \Downarrow \sigma'} \text{ IF-F}$$

$$\frac{\langle \sigma, b \rangle \Downarrow \text{false}}{\langle \sigma, \text{while } b \text{ do } c \rangle \Downarrow \sigma} \text{ WHILE-F}$$

$$\frac{\langle \sigma, b \rangle \Downarrow \text{true} \quad \langle \sigma, c \rangle \Downarrow \sigma' \quad \langle \sigma', \text{while } b \text{ do } c \rangle \Downarrow \sigma''}{\langle \sigma, \text{while } b \text{ do } c \rangle \Downarrow \sigma''} \text{ WHILE-T}$$

Note:

The big-step semantics treat termination differently to small-step. To prove these two are equal, we could assume termination.

It's interesting to see that the rule for while loops does not rely on using an if command (as in the case of small-step semantics). Why does this rule work?

1.4.1 Command equivalence

The small-step operational semantics suggests that the loop **while** b **do** c should be equivalent to the command **if** b **then** $(c; \text{while } b \text{ do } c)$ **else skip**. Can we show that this indeed the case that the language is defined using the above large-step evaluation?

First, we need to be more precise about what “equivalent commands” mean. Our formal model allows us to define this concept using large-step evaluations as follows. (One can write a similar definition using $\rightarrow^*$ in small-step semantics.)

Definition 1.4.1: Equivalence of commands

Two commands c and c' are equivalent (written $c \sim c'$) if, for any stores σ and σ' , we have

$$\langle \sigma, c \rangle \Downarrow \sigma' \iff \langle \sigma, c' \rangle \Downarrow \sigma'.$$

We can now state and prove the claim that **while** b **do** c and **if** b **then** $(c; \text{while } b \text{ do } c)$ **else skip** are equivalent.

Theorem 1.4.1

For all $b \in \mathbf{Bexp}$ and $c \in \mathbf{Com}$ we have

$$\text{while } b \text{ do } c \sim \text{if } b \text{ then } (c; \text{while } b \text{ do } c) \text{ else skip}.$$

: Let W be an abbreviation for **while** b **do** c . We want to show that for all stores σ, σ' , we have:

$$\langle \sigma, W \rangle \Downarrow \sigma' \quad \text{if and only if} \quad \langle \sigma, \text{if } b \text{ then } (c; W) \text{ else skip} \rangle \Downarrow \sigma'$$

For this, we must show that both directions ($\implies$ and $\impliedby$) hold. We'll show only direction $\implies$; the other is similar.

Assume that σ and σ' are stores such that $\langle \sigma, W \rangle \Downarrow \sigma'$. It means that there is some derivation that proves this fact. Inspecting the evaluation rules, we see that there are two possible rules whose conclusions match this fact: WHILE-F and WHILE-T. We analyze each of them in turn.

While-F. The derivation must look like the following.

$$\frac{\langle \sigma, b \rangle \overset{\vdots_1}{\Downarrow} \text{false}}{\langle \sigma, W \rangle \Downarrow \sigma} \text{ WHILE-F}$$

Here, we use $\vdots_1$ to refer to the derivation of $\langle \sigma, b \rangle \Downarrow \text{false}$. Note that in this case, $\sigma' = \sigma$.

We can use $\vdots_1$ to derive a proof tree showing that the evaluation of **if** b **then** $(c; W)$ **else skip** yields the same final state σ :

$$\frac{\langle \sigma, b \rangle \overset{\vdots_1}{\Downarrow} \text{false} \quad \frac{}{\langle \sigma, \text{skip} \rangle \Downarrow \sigma} \text{ SKIP}}{\langle \sigma, \text{if } b \text{ then } (c; W) \text{ else skip} \rangle \Downarrow \sigma} \text{ IF-F}$$

While-T. In this case, the derivation has the following form.

$$\frac{\langle \sigma, b \rangle \overset{\vdots_2}{\Downarrow} \text{true} \quad \langle \sigma, c \rangle \overset{\vdots_3}{\Downarrow} \sigma'' \quad \langle \sigma'', W \rangle \overset{\vdots_4}{\Downarrow} \sigma'}{\langle \sigma, W \rangle \Downarrow \sigma'} \text{ WHILE-T}$$

We can use subderivations $\vdots_2$, $\vdots_3$, and $\vdots_4$ to show that the evaluation of **if** b **then** $(c; W)$ **else skip** yields the same final state σ' .

$$\frac{\langle \sigma, b \rangle \overset{\vdots_2}{\Downarrow} \text{true} \quad \frac{\langle \sigma, c \rangle \overset{\vdots_3}{\Downarrow} \sigma'' \quad \langle \sigma'', W \rangle \overset{\vdots_4}{\Downarrow} \sigma'}{\langle \sigma, c; W \rangle \Downarrow \sigma'} \text{ SEQ}}{\langle \sigma, \text{if } b \text{ then } (c; W) \text{ else skip} \rangle \Downarrow \sigma'} \text{ IF-T}$$

Hence, we showed that in each of the two possible cases, the command **if** b **then** $(c; W)$ **else skip** evaluates to the same final state as the command W . QED

Some questions:

- Can we write a program that doesn't terminate? yes, obv
- Is IMP Turing Complete? Kinda, we don't have a way of creating new state (it's statically allocated), so if we replace the unbound **Int** type with something like **Int64**, it wouldn't be complete

Chapter 2

Denotational Semantics

We have now seen two operational models for programming languages: small-step and large-step. In this lecture, we consider a different semantic model, called *denotational semantics*.

The idea in denotational semantics is to express the meaning of a program as the mathematical function that expresses what the program computes. We can think of an IMP program c as a function from stores to stores: given an initial store, the program produces a final store. For example, the program $foo := bar + 1$ can be thought of as a function that when given an input store σ , produces a final store σ' that is identical to σ except that it maps foo to the integer $\sigma(bar) + 1$; that is, $\sigma' = \sigma[foo \mapsto \sigma(bar) + 1]$. We will model programs as functions from input stores to output stores. As opposed to operational models, which tell us *how* programs execute, the denotational model shows us *what* programs compute.

2.1 A Denotational Semantics for IMP

For each program c , we write $C[[c]]$ for the *denotation* of c , that is, the mathematical function that c represents:

$$C[[c]] : \text{Store} \rightarrow \text{Store}.$$

Note that $C[[c]]$ is actually a partial function (as opposed to a total function), both because the store may not be defined on the free variables of the program and because the program may not terminate for certain input stores.

The function $C[[c]]$ is not defined for non-terminating programs as they have no corresponding output stores.

We will write $C[[c]]\sigma$ for the result of applying the function $C[[c]]$ to the store σ . That is, if f is the function that $C[[c]]$ denotes, then we write $C[[c]]\sigma$ to mean the same thing as $f(\sigma)$.

We must also model expressions as functions, this time from stores to the values they represent. We will write $\mathcal{A}[[a]]$ for the denotation of arithmetic expression a , and $\mathcal{B}[[b]]$ for the denotation of boolean expression b .

$$\mathcal{A}[[a]] : \text{Store} \rightarrow \text{Int} \quad \mathcal{B}[[b]] : \text{Store} \rightarrow \{\text{true}, \text{false}\}$$

Now we want to define these functions. To make it easier to write down these definitions, we will describe (partial) functions using sets of pairs. More precisely, we will represent a partial map $f : A \rightarrow B$ as a set of pairs $F = \{(a, b) \mid a \in A \text{ and } b = f(a) \in B\}$ such that, for each $a \in A$, there is at most one pair of the form $(a, _)$ in the set. Hence $(a, b) \in F$ is the same as $b = f(a)$.

We can now define denotations for IMP. We start with the denotations of expressions:

$$\begin{aligned} \mathcal{A}[[n]] &= \{(\sigma, n)\} \\ \mathcal{A}[[x]] &= \{(\sigma, \sigma(x))\} \\ \mathcal{A}[[a_1 + a_2]] &= \{(\sigma, n) \mid (\sigma, n_1) \in \mathcal{A}[[a_1]] \wedge (\sigma, n_2) \in \mathcal{A}[[a_2]] \wedge n = n_1 + n_2\} \\ \mathcal{B}[[\text{true}]] &= \{(\sigma, \text{true})\} \\ \mathcal{B}[[\text{false}]] &= \{(\sigma, \text{false})\} \\ \mathcal{B}[[a_1 < a_2]] &= \{(\sigma, \text{true}) \mid (\sigma, n_1) \in \mathcal{A}[[a_1]] \wedge (\sigma, n_2) \in \mathcal{A}[[a_2]] \wedge n_1 < n_2\} \\ &\quad \cup \{(\sigma, \text{false}) \mid (\sigma, n_1) \in \mathcal{A}[[a_1]] \wedge (\sigma, n_2) \in \mathcal{A}[[a_2]] \wedge n_1 \geq n_2\} \end{aligned}$$

The denotations for commands are as follows:

$$\begin{aligned} C[\text{skip}] &= \{(\sigma, \sigma)\} \\ C[x := a] &= \{(\sigma, \sigma[x \mapsto n]) \mid (\sigma, n) \in \mathcal{A}[a]\} \\ C[c_1; c_2] &= \{(\sigma, \sigma') \mid \exists \sigma''. ((\sigma, \sigma'') \in C[c_1] \wedge (\sigma'', \sigma') \in C[c_2])\} \end{aligned}$$

All these can be expressed in function-style notation too.

Note:

If the denotation of σ under b is undefined, then the function should also be undefined, hence the case completeness.

Note that $C[c_1; c_2] = C[c_2] \circ C[c_1]$, where $\circ$ is the composition of relations, defined as follows: if $R_1 \subseteq A \times B$ and $R_2 \subseteq B \times C$ then $R_2 \circ R_1 \subseteq A \times C$ is $R_2 \circ R_1 = \{(a, c) \mid \exists b \in B. (a, b) \in R_1 \wedge (b, c) \in R_2\}$. If $C[c_1]$ and $C[c_2]$ are total functions, then $\circ$ is function composition.

$$\begin{aligned} C[\text{if } b \text{ then } c_1 \text{ else } c_2] &= \{(\sigma, \sigma') \mid (\sigma, \text{true}) \in \mathcal{B}[b] \wedge (\sigma, \sigma') \in C[c_1]\} \\ &\quad \cup \{(\sigma, \sigma') \mid (\sigma, \text{false}) \in \mathcal{B}[b] \wedge (\sigma, \sigma') \in C[c_2]\} \end{aligned}$$

$$\begin{aligned} C[\text{while } b \text{ do } c] &= \{(\sigma, \sigma) \mid (\sigma, \text{false}) \in \mathcal{B}[b]\} \\ &\quad \cup \{(\sigma, \sigma') \mid (\sigma, \text{true}) \in \mathcal{B}[b] \wedge \exists \sigma''. ((\sigma, \sigma'') \in C[c] \wedge (\sigma'', \sigma') \in C[\text{while } b \text{ do } c])\} \end{aligned}$$

But now we have a problem: the last “definition” is not really a definition because it expresses $C[\text{while } b \text{ do } c]$ in terms of itself! This is not a definition but a recursive equation. What we want is the solution to this equation.

Note:

The second invocation of while isn’t completely circular because the state changes.

2.2 Fixed Points

We gave a recursive equation that the function $C[\text{while } b \text{ do } c]$ must satisfy. To understand some of the issues involved, let’s consider a simpler example. Consider the following equation for a function $f : \mathbb{N} \rightarrow \mathbb{N}$.

$$f(x) = \begin{cases} 0 & \text{if } x = 0 \\ f(x-1) + 2x - 1 & \text{otherwise} \end{cases} \quad (2.1)$$

This is not a definition for f , but rather an equation that we want f to satisfy. What function, or functions, satisfy this equation for f ? The only solution to this equation is the function $f(x) = x^2$.

In general, there may be no solutions for a recursive equation (e.g., there are no functions $g : \mathbb{N} \rightarrow \mathbb{N}$ that satisfy the recursive equation $g(x) = g(x) + 1$), or multiple solutions (e.g., find two functions $g : \mathbb{R} \rightarrow \mathbb{R}$ that satisfy $g(x) = 4 \times g(\frac{x}{2})$).

We can compute solutions to such equations by building successive approximations. Each approximation is closer and closer to the solution. To solve the recursive equation for f , we start with the partial function $f_0 = \emptyset$ (i.e., f_0 is the empty relation; it is a partial function with the empty set for its domain). We compute successive

approximations using the recursive equation.

$$\begin{aligned}
f_0 &= \emptyset \\
f_1 &= \begin{cases} 0 & \text{if } x = 0 \\ f_0(x-1) + 2x - 1 & \text{otherwise} \end{cases} = \{(0, 0)\} \\
f_2 &= \begin{cases} 0 & \text{if } x = 0 \\ f_1(x-1) + 2x - 1 & \text{otherwise} \end{cases} = \{(0, 0), (1, 1)\} \\
f_3 &= \begin{cases} 0 & \text{if } x = 0 \\ f_2(x-1) + 2x - 1 & \text{otherwise} \end{cases} = \{(0, 0), (1, 1), (2, 4)\} \\
&\vdots
\end{aligned}$$

This sequence of successive approximations f_i gradually builds the function $f(x) = x^2$.

We can model this process of successive approximations using a higher-order function F that takes one approximation f_k and returns the next approximation f_{k+1} :

$$F : (\mathbb{N} \rightarrow \mathbb{N}) \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$$

where

$$(F(f))(x) = \begin{cases} 0 & \text{if } x = 0 \\ f(x-1) + 2x - 1 & \text{otherwise} \end{cases}$$

A solution to the recursive equation (1) is a function f such that $f = F(f)$. In general, given a function $F : A \rightarrow A$, we have that $a \in A$ is a *fixed point* of F if $F(a) = a$. We also write $a = \text{fix}(F)$ to indicate that a is a least fixed point of F .

So the solution to the recursive equation (1) is a fixed-point of the higher-order function F . We can compute this fixed point iteratively, starting with $f_0 = \emptyset$ and at each iteration computing $f_{k+1} = F(f_k)$. The fixed point is the limit of this process:

$$\begin{aligned}
f &= \text{fix}(F) \\
&= f_0 \cup f_1 \cup f_2 \cup f_3 \cup \dots \\
&= \emptyset \cup F(\emptyset) \cup F(F(\emptyset)) \cup F(F(F(\emptyset))) \cup \dots \\
&= \bigcup_{i \geq 0} F^i(\emptyset)
\end{aligned}$$

2.3 Denotation for Loops

We can now write the correct denotation case for while loops as the fixed point of a higher-order function:

$$C[\text{while } b \text{ do } c] = \text{fix}(F)$$

where

$$F(f) = \{(\sigma, \sigma) \mid (\sigma, \text{false}) \in \mathcal{B}[b]\} \cup \{(\sigma, \sigma') \mid (\sigma, \text{true}) \in \mathcal{B}[b] \wedge \exists \sigma''. (\sigma, \sigma'') \in C[c] \wedge (\sigma'', \sigma') \in f\}$$

2.4 Kleene's Fixed Point Theorem

Next we'll prove Kleene's fixpoint theorem, which shows that the fixed point used to define the semantics of while commands exists.

Definition 2.4.1: Scott Continuity

A function F from U to U is said to be *Scott-continuous* if for every chain $X_1 \subseteq X_2 \subseteq \dots$ we have $F(\bigcup_i X_i) = \bigcup_i F(X_i)$.

It is not hard to show that if F is Scott continuous, then it is also *monotonic*—that is, $X \subseteq Y$ implies $F(X) \subseteq F(Y)$. The proof of this fact is left as an exercise.

Theorem 2.4.1 Kleene Fixpoint

Let F be a Scott-continuous function. The least fixed point of F is $\bigcup_i F^i(\emptyset)$.

: Let $X = \bigcup_i F^i(\emptyset)$.

First, we will prove that X is a fixed point of F —that is, $F(X) = X$. We calculate as follows:

$$\begin{aligned}
 F(X) &= F\left(\bigcup_i F^i(\emptyset)\right) && \text{By definition of } X \\
 &= \bigcup_i F(F^i(\emptyset)) && \text{By Scott continuity} \\
 &= \bigcup_i F^{i+1}(\emptyset) \\
 &= \emptyset \cup \bigcup_i F^{i+1}(\emptyset) \\
 &= F^0(\emptyset) \cup \bigcup_i F^{i+1}(\emptyset) \\
 &= \bigcup_i F^i(\emptyset) \\
 &= X
 \end{aligned}$$

Second, we will show that X is the least fixed point of F . Suppose that Y is some other arbitrary fixed point of F . By induction, we can easily show that $F^i(\emptyset) \subseteq Y$ for all i . For the base case, i is 0 and we trivially have $F^0(\emptyset) = \emptyset \subseteq Y$. For the inductive case, we assume that $F^i(\emptyset) \subseteq Y$ and prove that $F^{i+1}(\emptyset) \subseteq Y$. By our inductive hypothesis and the fact that F is monotone, we have that $F(F^i(\emptyset)) \subseteq F(Y)$. As Y is a fixed point we also have $F(Y) = Y$ and so $F^{i+1}(\emptyset) \subseteq Y$.

Then, since every element of the chain

$$F^0(\emptyset) \subseteq F^1(\emptyset) \subseteq \dots$$

is a subset of Y , immediately we have that their union, $X = \bigcup_i F^i(\emptyset) \subseteq Y$. Hence, X is the least (with respect to $\subseteq$) fixed point of F . Q

2.5 Reasoning using Denotational Semantics

One of the main advantages of using denotational semantics compared to operational semantics is that proofs of equivalence can be carried out directly by simply calculating the denotations of programs and then arguing that they are identical. This is in contrast to operational techniques, where one must reason explicitly about low-level transitions and derivations involving ad hoc abstract machines.

As an example, to show that **skip**; c and c ; **skip** are equivalent, we can calculate as follows,

$$\begin{aligned}
 C[\llbracket \text{skip}; c \rrbracket] &= \{(\sigma, \sigma'') \mid \exists \sigma'. (\sigma, \sigma') \in C[\llbracket \text{skip} \rrbracket] \wedge (\sigma', \sigma'') \in C[\llbracket c \rrbracket]\} \\
 &= \{(\sigma, \sigma'') \mid (\sigma, \sigma'') \in C[\llbracket c \rrbracket]\} \\
 &= \{(\sigma, \sigma'') \mid \exists \sigma'. (\sigma, \sigma') \in C[\llbracket c \rrbracket] \wedge (\sigma', \sigma'') \in C[\llbracket \text{skip} \rrbracket]\} \\
 &= C[\llbracket c; \text{skip} \rrbracket]
 \end{aligned}$$

using standard facts about partial functions, relations, and sets as convenient in the proof itself.

Next, consider the command $C[\llbracket \text{while false do } c \rrbracket]$. By the definition of the denotational semantics, this is equal to $\text{fix}(F)$ where

$$F(f) = \{(\sigma, \sigma) \mid (\sigma, \text{false}) \in \mathcal{B}[\llbracket b \rrbracket]\} \cup \{(\sigma, \sigma'') \mid (\sigma, \text{true}) \in \mathcal{B}[\llbracket b \rrbracket] \wedge (\sigma, \sigma') \in C[\llbracket c \rrbracket] \wedge (\sigma', \sigma'') \in f\}$$

By the Kleene fixpoint theorem we have that $\text{fix}(F) = \bigcup_i F^i(\emptyset)$. It is straightforward to show by induction that since the guard is false, for all i we have $F^i(\emptyset) = \{(\sigma, \sigma)\}$. It follows that $\text{fix}(F) = \{(\sigma, \sigma)\}$, which is just $C[\llbracket \text{skip} \rrbracket]$.

As an exercise, calculate $C[\llbracket \text{while true do skip} \rrbracket]$ using the same technique.

Chapter 3

Axiomatic Semantics

3.1 Introduction to axiomatic semantics

Now we turn to the third and final main style of semantics, *axiomatic semantics*. The idea in axiomatic semantics is to define meaning in terms of logical specifications that programs satisfy. This is in contrast to operational models (which show *how* programs execute) and denotational models (which show *what* programs compute). This approach to reasoning about programs and expressing program semantics was originally proposed by Floyd and Hoare and was developed further by Dijkstra and Gries.

A common way to express program specifications is in terms of pre-conditions and post-conditions:

$$\{Pre\} c \{Post\}$$

where c is a program, and Pre and $Post$ are formulas that describe properties of the program state, usually referred to as *assertions*. Such a triple is usually referred to as a *partial correctness specification* (or sometimes a “Hoare triple”) and has the following meaning:

“If Pre holds before executing c , and c terminates, then $Post$ holds after c .”

In other words, if we start with a store σ in which Pre holds and the execution of c with respect to σ terminates and yields a store σ' , then $Post$ holds in store σ' .

Pre-conditions and post-conditions can be regarded as interfaces or contracts between the program and its clients. They help users to understand what the program is supposed to yield without needing to understand how the program executes. Typically, programmers write them as comments for procedures and functions as documentation and to make it easier to maintain programs. Such specifications are especially useful for library functions for which the source code is often not available to the users. In this case, pre-conditions and post-conditions serve as contracts between the library developers and users of the library.

However, there is no guarantee that pre-conditions and post-conditions written informally in comments are correct: the comments describe the intent of the developer, but they do not give a guarantee of correctness. Axiomatic semantics addresses this problem. It shows how to rigorously describe partial correctness statements and how to establish correctness using formal reasoning.

Note that partial correctness specifications don’t ensure that the program will terminate—this is why they are called “partial”. In contrast, *total correctness statements* ensure that the program terminates whenever the precondition holds. Such statements are denoted using square brackets:

$$[Pre] c [Post]$$

meaning:

“If Pre holds before c then c will terminate and $Post$ will hold after c .”

In general a pre-condition specifies what the program expects before execution and the post-condition specifies what guarantees the program provides (if the program terminates). Here is a simple example:

$\{foo = 0 \wedge bar = i\} \text{ } baz := 0; \text{ while } foo \neq bar \text{ do } (baz := baz - 2; foo := foo + 1) \{baz = -2i\}$

It says that if the store maps foo to 0 and bar to i before execution, then, if the program terminates, the final store will map baz to $-2i$ (i.e., -2 times the initial value of bar). Note that i is a *logical variable* that doesn't occur in the program and is only used to express the initial value of bar . Such variables are sometimes called *ghost variables*.

This partial correctness statement is valid. That is, it is indeed the case that if we have any store σ such that $\sigma(foo) = 0$, and

$$C \llbracket baz := 0; \text{ while } foo \neq bar \text{ do } (baz := baz - 2; foo := foo + 1) \rrbracket \sigma = \sigma',$$

then $\sigma'(baz) = -2\sigma(bar)$.

Note that this is a partial correctness statement: if the pre-condition is true before c , and c terminates then the post-condition holds after c . There are some initial stores for which the program will not terminate.

The following total correctness statement is true.

$$[foo = 0 \wedge bar = i \wedge i \geq 0] \text{ baz := 0; while } foo \neq bar \text{ do } (baz := baz - 2; foo := foo + 1) [baz = -2i]$$

That is, if we start with a store σ that maps foo to 0 and bar to a non-negative integer, then the execution of the command will terminate in a final store σ' with $\sigma'(baz) = -2\sigma(bar)$.

The following partial correctness statement is not valid. (Why not?)

$$\{foo = 0 \wedge bar = i\} \text{ baz := 0; while } foo \neq bar \text{ do } (baz := baz + foo; foo := foo + 1) \{baz = i\}$$

In the rest of our discussion of axiomatic semantics we will focus almost exclusively on partial correctness assertions.

3.2 Assertions

Now we turn to the following issues:

- What logic do we use for writing assertions? That is, what can we express in pre-conditions and post-conditions?
- What does it mean that an assertion is valid? What does it mean that a partial correctness statement $\{Pre\} c \{Post\}$ is valid?
- How can we prove that a partial correctness statement is valid?

What can we say in pre-conditions and post-conditions? In the examples so far, we have used program variables, equality, logical variables (e.g., i), and conjunction ($\wedge$). What we allow in pre-conditions and post-conditions directly influences the sorts of program properties we can describe using partial correctness statements. We will use the set of logical formulas including comparisons between arithmetic expressions, standard logical operators (and, or, implication, negation), as well as quantifiers (universal and existential). Assertions may also introduce logical variables that are different than the variables appearing in the program.

$$i, j \in \mathbf{LVar}$$

$$a \in \mathbf{Aexp} ::= x \mid i \mid n \mid a_1 + a_2 \mid a_1 \times a_2$$

$$P, Q \in \mathbf{Assn} ::= \mathbf{true} \mid \mathbf{false} \mid a_1 < a_2 \mid P_1 \wedge P_2 \mid P_1 \vee P_2 \mid P_1 \Rightarrow P_2 \mid \neg P \mid \forall i. P \mid \exists i. P$$

Observe that the domain of boolean expressions $\mathbf{Bexp}$ is a subset of the domain of assertions. Notable additions over the syntax of boolean expressions are quantifiers ($\forall$ and $\exists$). For instance, one can express the fact that variable x divides variable y using existential quantification: $\exists i. x \times i = y$.

3.3 Satisfaction and Validity

Now we would like to describe what we mean by “assertion P holds in store σ ”. But to determine whether P holds or not, we need more than just the store σ (which maps program variables to their values); we also need to know the values of the logical variables. We describe those values using an *interpretation* I ,

$$I : \mathbf{LVar} \rightarrow \mathbf{Int},$$

and define the function $\mathcal{A}_i[\![a]\!]$, which is like the denotation of expressions extended to logical variables in the obvious way:

$$\begin{aligned}\mathcal{A}_i[\![n]\!](\sigma, I) &= n \\ \mathcal{A}_i[\![x]\!](\sigma, I) &= \sigma(x) \\ \mathcal{A}_i[\![i]\!](\sigma, I) &= I(i) \\ \mathcal{A}_i[\![a_1 + a_2]\!](\sigma, I) &= \mathcal{A}_i[\![a_1]\!](\sigma, I) + \mathcal{A}_i[\![a_2]\!](\sigma, I)\end{aligned}$$

Now we can express the satisfiability of assertions as a relation $\sigma \models_I P$ read as “ P is satisfied in store σ under interpretation I ,” or “store σ satisfies assertion P under interpretation I .” We will write $\sigma \not\models_I P$ whenever $\sigma \models_I P$ doesn’t hold.

$$\begin{aligned}\sigma \models_I \mathbf{true} & \quad (\text{always}) \\ \sigma \models_I a_1 < a_2 & \quad \text{if } \mathcal{A}_i[\![a_1]\!](\sigma, I) < \mathcal{A}_i[\![a_2]\!](\sigma, I) \\ \sigma \models_I P_1 \wedge P_2 & \quad \text{if } \sigma \models_I P_1 \text{ and } \sigma \models_I P_2 \\ \sigma \models_I P_1 \vee P_2 & \quad \text{if } \sigma \models_I P_1 \text{ or } \sigma \models_I P_2 \\ \sigma \models_I P_1 \Rightarrow P_2 & \quad \text{if } \sigma \not\models_I P_1 \text{ or } \sigma \models_I P_2 \\ \sigma \models_I \neg P & \quad \text{if } \sigma \not\models_I P \\ \sigma \models_I \forall i. P & \quad \text{if } \forall k \in \text{Int}. \sigma \models_{I[i \mapsto k]} P \\ \sigma \models_I \exists i. P & \quad \text{if } \exists k \in \text{Int}. \sigma \models_{I[i \mapsto k]} P\end{aligned}$$

We can now say that an assertion P is *valid* (written $\models P$) if it is valid in any store, under any interpretation: $\forall \sigma, I. \sigma \models_I P$.

Having defined validity for individual assertions, we now turn to partial correctness statements. We say that a partial correctness statement $\{P\} c \{Q\}$ is *satisfied* in store σ and interpretation I , written $\sigma \models_I \{P\} c \{Q\}$, if:

$$\forall \sigma'. \text{ if } \sigma \models_I P \text{ and } C[\![c]\!]\sigma = \sigma' \text{ then } \sigma' \models_I Q$$

Note that this definition depends on the execution of c in the initial store σ .

Finally, we can say that a partial correctness triple is *valid* (written $\models \{P\} c \{Q\}$), if it is valid in any store and interpretation:

$$\forall \sigma, I. \sigma \models_I \{P\} c \{Q\}.$$

Now we know what we mean when we say “assertion P holds” or “partial correctness statement $\{P\} c \{Q\}$ is valid.”

3.4 Hoare Logic

How do we show that a partial correctness statement $\{P\} c \{Q\}$ holds? We know that $\{P\} c \{Q\}$ is valid if it holds for all stores and interpretations: $\forall \sigma, I. \sigma \models_I \{P\} c \{Q\}$. Furthermore, showing that $\sigma \models_I \{P\} c \{Q\}$ requires reasoning about the execution of command c (that is, $C[\![c]\!]$), as indicated by the definition of validity.

It turns out that there is an elegant way of deriving valid partial correctness statements, without having to reason about stores, interpretations, and the execution of c . We can use a set of inference rules and axioms, called *Hoare rules*, to directly derive valid partial correctness statements. The set of rules forms a proof system known as *Hoare logic*.

$$\begin{array}{c} \frac{}{\{P\} \mathbf{skip} \{P\}} \text{ SKIP} \qquad \frac{}{\{P[a/x]\} x := a \{P\}} \text{ ASSIGN} \\[10pt] \frac{\{P\} c_1 \{R\} \quad \{R\} c_2 \{Q\}}{\{P\} c_1; c_2 \{Q\}} \text{ SEQ} \\[10pt] \frac{\{P \wedge b\} c_1 \{Q\} \quad \{P \wedge \neg b\} c_2 \{Q\}}{\{P\} \mathbf{if } b \mathbf{ then } c_1 \mathbf{ else } c_2 \{Q\}} \text{ IF} \\[10pt] \frac{\{P \wedge b\} c \{P\}}{\{P\} \mathbf{while } b \mathbf{ do } c \{P \wedge \neg b\}} \text{ WHILE} \end{array}$$

The assertion P in the rule for while loops is essentially a *loop invariant*; it is an assertion that holds before and after each iteration, as shown in the premise of the rule. Therefore, it is both a pre-condition for the loop (because it holds before the first iteration); and also a post-condition for the loop (because it holds after the last iteration). The fact that P is both a pre- and post-condition for the while loop is reflected in the conclusion of the rule.

There is one more rule, the rule of consequence, which allows to strengthen pre-conditions and weaken post-conditions:

$$\frac{\models (P \Rightarrow P') \quad \{P'\} c \{Q'\} \quad \models (Q' \Rightarrow Q)}{\{P\} c \{Q\}} \text{ CONSEQUENCE}$$

These set of Hoare rules represent an inductive definition for a set of partial correctness statements $\{P\} c \{Q\}$. We will say that $\{P\} c \{Q\}$ is a *theorem in Hoare logic*, written $\vdash \{P\} c \{Q\}$, if we can build a finite proof tree for it.

3.5 Soundness and Completeness

At this point we have two kinds of partial correctness assertions:

- valid partial correctness statements $\models \{P\} c \{Q\}$, which hold for all stores and interpretations, according to the semantics of c , and
- Hoare logic theorems $\vdash \{P\} c \{Q\}$, that is, partial correctness statements that can be derived using the axioms and rules of Hoare logic.

The question is how do these sets relate to each other? More precisely, we have to answer two questions. First, is each Hoare logic theorem guaranteed to be valid partial correctness triple? In other words,

$$\text{does } \vdash \{P\} c \{Q\} \text{ imply } \models \{P\} c \{Q\}?$$

The answer is yes, and it shows that Hoare logic is *sound*. Soundness is important because it says that Hoare logic doesn't allow us to derive partial correctness assertions that actually don't hold. The proof of soundness requires induction on the derivations in $\vdash \{P\} c \{Q\}$ (we omit this proof).

The second question refers to the expressiveness and power of Hoare rules: can we always build a Hoare logic proof for each valid assertion? In other words,

$$\text{does } \models \{P\} c \{Q\} \text{ imply } \vdash \{P\} c \{Q\}?$$

The answer is a qualified yes: if $\models \{P\} c \{Q\}$ then there is a proof of $\vdash \{P\} c \{Q\}$ using the rules of Hoare logic, provided there are proofs for the validity of assertions that occur in the rule of consequence $\models (P \Rightarrow P')$ and $\models (Q' \Rightarrow Q)$. This result is known as the *relative completeness* of Hoare logic and is due to Cook (1974).

3.6 Example: Factorial

As an example illustrating how we can use Hoare logic to verify the correctness of a program, consider a program that computes the factorial of a number n :

```

{x = n ∧ n > 0}
y := 1;
while x > 0 do (
  y := y * x;
  x := x - 1
)
{y = n!}

```

Because the derivation for this proof is somewhat large, we will go through the reasoning used to construct it step by step.

At the top level, the program is a sequence of an assignment and a loop. To use the SEQ rule, we need to find an assertion that holds after the assignment and before the loop. Examining the rule for while loops, we see that the assertion before the loop must be an invariant for the loop. Inspecting the loop we see that it builds the factorial up in y starting with n , then multiplying it by $n - 1$, then $n - 2$, etc. At each iteration, x contains the next value multiplied into y , that is:

$$y = n * (n - 1) * \dots * (x + 1)$$

If we multiply both sides of this equality by $x!$ and re-write the equality we get $x! * y = n!$, which is an invariant for the loop. However, to make the proof go through, we need a slightly stronger invariant:

$$I = x! * y = n! \wedge x \geq 0$$

Having identified a suitable loop invariant, let us take a step back and review where we are. We want to prove that our overall partial correctness specification is valid. To do this, we need to show two facts:

$$\{x = n \wedge n > 0\} y := 1 \{I\} \quad (3.1)$$

$$\{I\} \text{ while } x > 0 \text{ do } (y := y * x; x := x - 1) \{y = n!\} \quad (3.2)$$

After showing that both (1) and (2) hold, we can use the rule SEQ to obtain the desired result.

To show (1), we use the ASSIGN axiom and obtain the following: $\{I[1/y]\} y := 1 \{I\}$. Expanding this out, we obtain:

$$\{x! * 1 = n! \wedge x \geq 0\} y := 1 \{x! * y = n! \wedge x \geq 0\}$$

With the following implication,

$$x = n \wedge n > 0 \implies x! * 1 = n! \wedge x \geq 0,$$

(which can be shown by an easy calculation) we obtain (1) using the rule CONSEQUENCE.

Now let us prove (2). To use the WHILE rule, we need to show that I is an invariant for the loop:

$$\{I \wedge x > 0\} y := y * x; x := x - 1 \{I\} \quad (3.3)$$

We will show this by going backwards through the sequence of assignments:

$$\{(x - 1)! * y = n! \wedge (x - 1) \geq 0\} x := x - 1 \{I\} \quad (3.4)$$

$$\{(x - 1)! * y * x = n! \wedge (x - 1) \geq 0\} y := y * x \{(x - 1)! * y = n! \wedge (x - 1) \geq 0\} \quad (3.5)$$

Then, using the following implication:

$$I \wedge x > 0 \implies (x - 1)! * y * x = n! \wedge (x - 1) \geq 0$$

we obtain (3) using CONSEQUENCE, (4), and (5). Thus, I is an invariant for the loop and so by WHILE we obtain,

$$\{I\} \text{ while } x > 0 \text{ do } (y := y * x; x := x - 1) \{I \wedge x \leq 0\}$$

To finish the proof, we just have to show

$$I \wedge x \leq 0 \implies y = n!$$

i.e.,

$$x! * y = n! \wedge x \geq 0 \wedge x \leq 0 \implies y = n!$$

which holds as $x \geq 0$ and $x \leq 0$ implies $x = 0$ and so $x! = 1$. The result follows by CONSEQUENCE.

Chap ? in winscal has more info about this work plz

3.7 Decorated Programs

Doing a complete proof in Hoare Logic can feel overly verbose. The “core” of a proof consists of the preconditions and postconditions surrounding every command; with those, it’s usually possible to infer the complete shape of the proof tree.

Decorating programs is a way to informally write down a Hoare logic proof of correctness. The idea is to insert assertion decorations between every “line” of the program. Using this informal evidence, you can reconstruct the full formal proof.

3.7.1 Informal Rules

We’ll set out a few rules to check whether a decorated program represents a valid proof using *local consistency* checks. The rules, as you’ll see, are informal reflections of the formal inference rules for Hoare logic.

Skip. For **skip**, the precondition and postcondition should be the same, like this:

$$\begin{array}{c} \{P\} \\ \mathbf{skip} \\ \{P\} \end{array}$$

Sequence. For sequences, a new assertion R appears between the two commands. The two halves $\{P\} \ c_1 \ \{R\}$ and $\{R\} \ c_2 \ \{Q\}$ must be (recursively) locally consistent:

$$\begin{array}{c} \{P\} \\ c_1; \\ \{R\} \\ c_2 \\ \{Q\} \end{array}$$

Assignment. Assignments are locally consistent when the precondition is the same as the postcondition except that it substitutes the assigned expression in for the variable:

$$\begin{array}{c} \{P[a/x]\} \\ x := a \\ \{P\} \end{array}$$

Conditions. An **if** is locally consistent when both branches are locally consistent after adding the branch condition to each:

$$\begin{array}{c} \{P\} \\ \mathbf{if} \ b \ \mathbf{then} \\ \quad \{P \wedge b\} \\ \quad c_1 \\ \quad \{Q\} \\ \mathbf{else} \\ \quad \{P \wedge \neg b\} \\ \quad c_2 \\ \quad \{Q\} \\ \{Q\} \end{array}$$

Loops. A **while** command should be decorated with a loop invariant:

$$\begin{array}{c} \{P\} \\ \mathbf{while} \ b \ \mathbf{do} \\ \quad \{P \wedge b\} \\ \quad c \\ \quad \{P\} \\ \{P \wedge \neg b\} \end{array}$$

Implication. To capture the CONSEQUENCE rule, you can always write a (valid) implication to connect two commands:

$$\begin{array}{c} \{P\} \Rightarrow \\ \{Q\} \end{array}$$

Note:

It's possible to give a less redundant definition of these rules (less decorated), but this won't have the property of having a pre and post condition for every command.

3.7.2 Decorating a Program

These informal rules tell you how to verify whether a decorated program is correct, but they don't by themselves tell you how to construct those decorations (and thereby a proof). You can do this almost mechanically—except for loop invariants, which still require some creativity to concoct.

Here's an example program:

while (0 < y) **do** (x := x + 1; y := y - 1)

The first step is to decide what we want to prove about this program. So we write a precondition and postcondition for the whole program. Intuitively, the program adds y onto the initial value of the variable x, so we'll assert that:

$$\begin{array}{l} \{x = m \wedge y = n \wedge 0 \leq n\} \\ \mathbf{while} \ (0 < y) \ \mathbf{do} \ (\\ \quad x := x + 1; \\ \quad y := y - 1 \\ \quad) \\ \{x = m + n\} \end{array}$$

This program is a while command, so the next step is to come up with a loop invariant. We'll set up the structure first. We need an assertion I where we can “connect” the loop's precondition and postcondition to our overall precondition and postcondition using implications, like this:

$$\begin{array}{l} \{x = m \wedge y = n \wedge 0 \leq n\} \Rightarrow \\ \{I\} \\ \mathbf{while} \ (0 < y) \ \mathbf{do} \ (\\ \quad \{I \wedge 0 < y\} \\ \quad x := x + 1; \\ \quad y := y - 1 \\ \quad \{I\} \\ \quad) \\ \{I \wedge 0 \not< y\} \Rightarrow \\ \{x = m + n\} \end{array}$$

On every iteration of the loop, the variable x is the sum we want, $m + n$, less the current value of y, which is the number of iterations remaining. So we'll define the invariant I like this:

$$I ::= (x = m + n - y) \wedge 0 \leq y$$

The top implication follows because $n - y = 0$, and the bottom implication is valid because $0 \not< y$ and $0 \leq y$ together imply $y = 0$.

To finish decorating the program, we need an assertion between the two lines in the body of the loop. By the rule for assignments, we know that this must be $I[y - 1/y]$, or:

$$P_1 ::= (x = m + n - (y - 1)) \wedge (0 \leq y - 1)$$

To make the assignment to x locally consistent, then, its precondition must be $P_1[x + 1/x]$, or:

$$P_2 ::= (x + 1 = m + n - (y - 1)) \wedge (0 \leq y - 1)$$

It's straightforward to see that the precondition at the top of the loop, $I \wedge 0 < y$, implies this new assertion P_2 . Now we can write our complete decorated program using these definitions:

$$\begin{array}{l}
\{x = m \wedge y = n \wedge 0 \leq n\} \Rightarrow \\
\{I\} \\
\text{while } (0 < y) \text{ do } (\\
\quad \{I \wedge 0 < y\} \Rightarrow \\
\quad \{P_2\} \\
\quad x := x + 1; \\
\quad \{P_1\} \\
\quad y := y - 1 \\
\quad \{I\} \\
) \\
\{I \wedge 0 \not< y\} \Rightarrow \\
\{x = m + n\}
\end{array}$$

3.8 Weakest Preconditions

Previously, we discussed the issue of completeness—i.e., whether it is possible to derive every valid partial correctness specification using the axioms and rules of Hoare logic. Unfortunately, if treated as a pure deductive system, Hoare logic cannot be complete. To see why, consider the following partial correctness specifications:

$$\{\text{true}\} \text{ skip } \{P\} \quad \{\text{true}\} c \{\text{false}\}$$

The first is valid if and only if the assertion P is valid while the second is valid if and only if the command c does not halt.

It turns out that the culprit is the CONSEQUENCE rule,

$$\frac{\models (P \Rightarrow P') \quad \{P'\} c \{Q'\} \quad \models (Q' \Rightarrow Q)}{\{P\} c \{Q\}} \text{ CONSEQUENCE}$$

which includes two premises about the validity of implications between the assertions involved. Proving those implications for our two examples would require proving arbitrary logical predicates P and proving that arbitrary IMP programs c terminate. By Gödel’s incompleteness theorem, we know that there is no consistent mathematical proof system that can do the former. Therefore, Hoare logic cannot be complete in that sense.

On the other hand, Hoare logic does enjoy this property:

Theorem 3.8.1 Relative Completeness of Hoare Logic

$\forall P, Q \in \mathbf{Assn}, c \in \mathbf{Com}. \models \{P\} c \{Q\} \text{ implies } \vdash \{P\} c \{Q\}.$

This result states that Hoare logic is no more incomplete than the language of assertions—i.e., if we had an oracle that could decide the validity of assertions, then we could decide the validity of partial correctness specifications. In other words, for any valid Hoare triple, there exists a proof in Hoare logic—as long as you have some external way to decide the validity of the premises to the CONSEQUENCE rule.

Cook’s proof of relative completeness depends on the notion of *weakest liberal preconditions*. Given a command c and a postcondition Q , the weakest liberal precondition is the weakest assertion P such that $\{P\} c \{Q\}$ is valid. Here, “weakest” means that any other valid precondition implies P . That is, P most accurately describes input states for which c either does not terminate or ends up in a state satisfying Q .

Definition 3.8.1: Weakest Liberal Precondition

Formally, an assertion P is a weakest liberal precondition of c and Q if:

$$\forall \sigma, I. \sigma \models_I P \iff (C \llbracket c \rrbracket \sigma) \text{ is undefined } \vee (C \llbracket c \rrbracket \sigma) \models_I Q$$

Note:

The interpretation is the same to check P and Q , meaning the program can’t change the set of logical variables.

We write $\text{wlp}(c, Q)$ for the weakest liberal precondition of command c and postcondition Q . From left-to-right, the formula above states that $\text{wlp}(c, Q)$ is a valid precondition: $\models \{P\} c \{Q\}$. The right-to-left implication says it

is the weakest valid precondition: if another assertion R satisfies $\models \{R\} c \{Q\}$, then R implies P . It can be shown that weakest liberal preconditions are unique modulo equivalence.

We can calculate the weakest liberal precondition of a command as follows:

$$\begin{aligned} \text{wlp}(\text{skip}, P) &= P \\ \text{wlp}(x := a, P) &= P[a/x] \\ \text{wlp}(c_1; c_2, P) &= \text{wlp}(c_1, \text{wlp}(c_2, P)) \\ \text{wlp}(\text{if } b \text{ then } c_1 \text{ else } c_2, P) &= (b \implies \text{wlp}(c_1, P)) \wedge (\neg b \implies \text{wlp}(c_2, P)) \end{aligned}$$

The definition of $\text{wlp}(\text{while } b \text{ do } c, P)$ is slightly more complicated—it encodes the weakest liberal precondition for each iteration of the loop. To give the intuition, first define the weakest liberal precondition for a loop that terminates in i steps as follows:

$$\begin{aligned} F_0(P) &= \text{true} \\ F_{i+1}(P) &= (\neg b \implies P) \wedge (b \implies \text{wlp}(c, F_i(P))) \end{aligned}$$

We can then express the weakest liberal precondition using an infinitary conjunction:

$$\text{wlp}(\text{while } b \text{ do } c, P) = \bigwedge_i F_i(P)$$

See Winskel Chapter 7 for the details of how to encode the weakest liberal precondition for a while loop as an ordinary assertion.

To check that our definition is correct, we can prove (how?) that it yields a valid partial correctness specification:

Lemma 3.8.1 Correctness of Weakest Preconditions

$$\forall c \in \mathbf{Com}, Q \in \mathbf{Assn}. \models \{\text{wlp}(c, Q)\} c \{Q\} \text{ and } \forall R \in \mathbf{Assn}. \models \{R\} c \{Q\} \text{ implies } (R \implies \text{wlp}(c, Q))$$

It is not hard to prove that it also yields a provable specification:

Lemma 3.8.2 Provability of Weakest Preconditions

$$\forall c \in \mathbf{Com}, Q \in \mathbf{Assn}. \vdash \{\text{wlp}(c, Q)\} c \{Q\}$$

Relative completeness follows by a simple argument:

Sketch: Let c be a command and let P and Q be assertions such that the partial correctness specification $\{P\} c \{Q\}$ is valid. By Lemma 1 we have $\models P \implies \text{wlp}(c, Q)$. By Lemma 2 we have $\vdash \{\text{wlp}(c, Q)\} c \{Q\}$. We conclude $\vdash \{P\} c \{Q\}$ using the CONSEQUENCE rule. $\square$